Game Spot Reservations: Service-Oriented Architecture with Two Providers and a Web Consumer

Cristian Farid Amado Peña and Cristian Santiago Silva Muñoz

Department of Systems Engineering, Universidad Santo Tomás, Villavicencio

Service-Oriented Development (Desarrollo Orientado a Servicios)

Diana Velásquez

November 18, 2026

Abstract

Game Spot is a video game lounge in the San Jorge neighborhood of Villavicencio, Colombia, that rents consoles by the hour and by the half hour. This document presents the service-oriented version of its booking system, built as three independent Spring Boot applications that communicate over HTTP with JSON. Provider 1 (port 8091) exposes the catalog of stations, their hourly and half-hour rates, and the opening hours. Provider 2 (port 8092) manages bookings in 30-minute blocks and issues JSON Web Tokens. The consumer application (port 8080) serves a web page written in HTML, CSS, and JavaScript with the Fetch API, and its backend calls both providers with Spring RestClient to coordinate two integrated operations: the daily availability of a station and the creation of a booking. Each provider persists its data in its own H2 file database defined with SQL scripts and accessed through JPA repositories. The document describes the business rules, the architecture, the API contract, the data transfer objects, the data model, the authentication design, and the tests: 29 automated tests across the three projects, a Postman collection of 40 requests, and a manual end-to-end test that includes a stopped provider.

Keywords: service-oriented architecture, REST API, Spring Boot, RestClient, JSON Web Token, H2

Game Spot Reservations: Service-Oriented Architecture with Two Providers and a Web Consumer

Game Spot rents five stations: two Xbox 360 consoles, two Xbox Series S consoles, and a VIP room with an Xbox Series X. Each type of station has an hourly rate and a separate half-hour rate: 4,000 and 2,500 Colombian pesos for the Xbox 360, 5,000 and 3,000 for the Xbox Series S, and 10,000 and 6,000 for the VIP room. Customers used to book through WhatsApp, so the lounge had no reliable way to show free time slots, to prevent two people from booking the same station at the same time, or to know who had paid. The team had already built a monolithic version of the system for another course; this project adapts the same business to the requirements of service-oriented development.

Splitting the system into services is justified because the two capabilities change for different reasons and at different times. The catalog changes when the owner buys a station, repairs one, or updates prices, and it is mostly read. Bookings change every time a customer reserves, pays, or cancels, and they carry the customer data and the authentication. Separating them lets each provider own its data and evolve independently, and the consumer shows how a client application combines both through their public contracts.

Objectives

The general objective is to build a web solution for booking Game Spot stations, organized as two provider applications and one consumer application that communicate through REST APIs. The specific objectives are to (a) expose the catalog of stations, rates, and opening hours as a provider with full CRUD operations, (b) expose bookings and authentication as a second provider, (c) build a consumer with a web interface that integrates both providers with RestClient, (d) persist each provider’s data in its own relational database with DDL, DML,

repositories, and ORM, (e) protect modifying operations with JWT, and (f) verify the solution with automated tests, Postman, and a test with a stopped provider.

Scope

The scope includes the three applications, the catalog CRUD, the booking life cycle (create, pay a simulated deposit, and cancel), the daily availability, the authentication of test users, and the web interface for customers and administrators. It excludes real online payments, e-mail notifications, user registration, deployment to a public server, a production database, and charging for extra time beyond the booked blocks. The last item was deliberately left for a later iteration, after the main booking flow, as agreed with the course instructor.

Business Rules

Table 1 lists the rules that the system applies and where each one is enforced. They were closed with the course instructor after the progress review: the lounge works with a single fixed schedule, the deposit is not refundable, an unpaid booking expires after 15 minutes, and extra time is handled after the main flow. The status column states whether the current version already enforces the rule.

Table 1

Business Rules of the Booking System

Rule	Definition	Where	Status
Opening hours	Every day from 3:00 p.m. to 9:00 p.m. (Bogotá time). A booking must start and end within those hours.	Provider 1 (horarios_atencion); consumer validation (400)	Implemented (CP-36g, CP-38d)
Blocks	Bookings start at :00 or :30 and last a multiple of 30 minutes, from 30 to 240 minutes, in the future.	Provider 2 (ReservaService, 400)	Implemented (CP-37c)
No overlaps	A station cannot have two active bookings (pending or confirmed) whose intervals overlap.	Provider 2 (JPQL query, 409)	Implemented (CP-37d)

Rule	Definition	Where	Status
Price	Each full hour at the hourly rate, plus the half-hour rate if 30 minutes remain. Xbox Series S: 30 min = 3,000; 60 min = 5,000; 90 min = 8,000.	Provider 2 (calcularPrecioPesos)	Implemented (CP-37b, CP-37i)
Deposit	50% of the price, rounded up. Paying it (simulated) changes the booking from PENDIENTE_PAGO to CONFIRMADA.	Provider 2	Implemented (CP-37f)
Non-refundable deposit	If a confirmed booking is cancelled, the deposit is not returned. The system has no refund operation: cancelling keeps the paid deposit on record.	Provider 2 (reservas.adelanto_pesos)	Implemented; the web page asks for confirmation
Logical cancellation	Bookings are never deleted: cancelling sets CANCELADA and the time, and frees the block.	Provider 2	Implemented (CP-37g)
Expiration	A booking that is not paid within 15 minutes of its creation expires (EXPIRADA): it frees the block and can no longer be paid or cancelled (409).	Provider 2 (ReservaService, buscarActivasQueSeCruzan)	Implemented (CP-37j, CP-37k)
Extra time	Charging for time played beyond the booked blocks.	—	Out of scope; after the main flow

Note. Test codes refer to Table 10.

The expiration is computed instead of stored. A booking keeps the state PENDIENTE_PAGO in the database, and Provider 2 treats it as expired when 15 minutes have passed since *creada_en*: the overlap and occupancy query only counts pending bookings created in the last 15 minutes, the responses show the state EXPIRADA, and paying or cancelling it answers 409. This way a booking expires exactly on time without a scheduled task, and the databases created before this rule did not need any change. Each pending booking also includes *pagarAntesDe*, the deadline to pay, which the web page shows to the customer. Before cancelling a confirmed booking, the page asks for confirmation and warns that the deposit is not refunded.

Users and Functionality

The solution has two kinds of users. A customer signs in, checks which 30-minute blocks of a station are free on a given day, books a block, pays the deposit, and cancels a booking. An

administrator signs in and changes the catalog, for example by taking a station out of service or changing a rate. Table 2 relates the functional requirements of the course guide to the place where the solution meets them.

Table 2

Functional Requirements and Where They Are Met

Code	Requirement	Implementation
RF-01	Solve the problem defined by the team	Online booking of Game Spot stations
RF-02	Provider 1 with a business capability	proveedor-catalogo: stations, rates, and opening hours
RF-03	Provider 2 with a different capability	proveedor-reservas: bookings and authentication
RF-04	Consumer connected to both providers	CatalogoCliente and ReservasCliente with RestClient
RF-05	At least two endpoints per provider	8 endpoints in Provider 1 and 7 in Provider 2
RF-06	GET, POST, and PUT, PATCH, or DELETE	All five methods are used
RF-07	Register information from the interface	Guided booking and sign-in form
RF-08	Query and show data in the interface	Stations, availability blocks, and the customer's bookings
RF-09	Update, cancel, or manage information	Pay the deposit, cancel a booking, take a station out of service
RF-10	Integrate both providers in one operation	Daily availability and booking creation
RF-11	Validate input and show errors	Bean Validation, business rules, and messages in the interface
RF-12	Test a stopped provider	503 response and a working page (Figures 7 and 8)
RF-13	CRUD on project resources	Full CRUD of stations; logical cancellation of bookings
RF-14	JWT authentication and a protected operation	Login in Provider 2; tokens required for bookings and catalog changes
RF-15	Persistence that survives a restart	H2 file databases; verified by restarting the providers

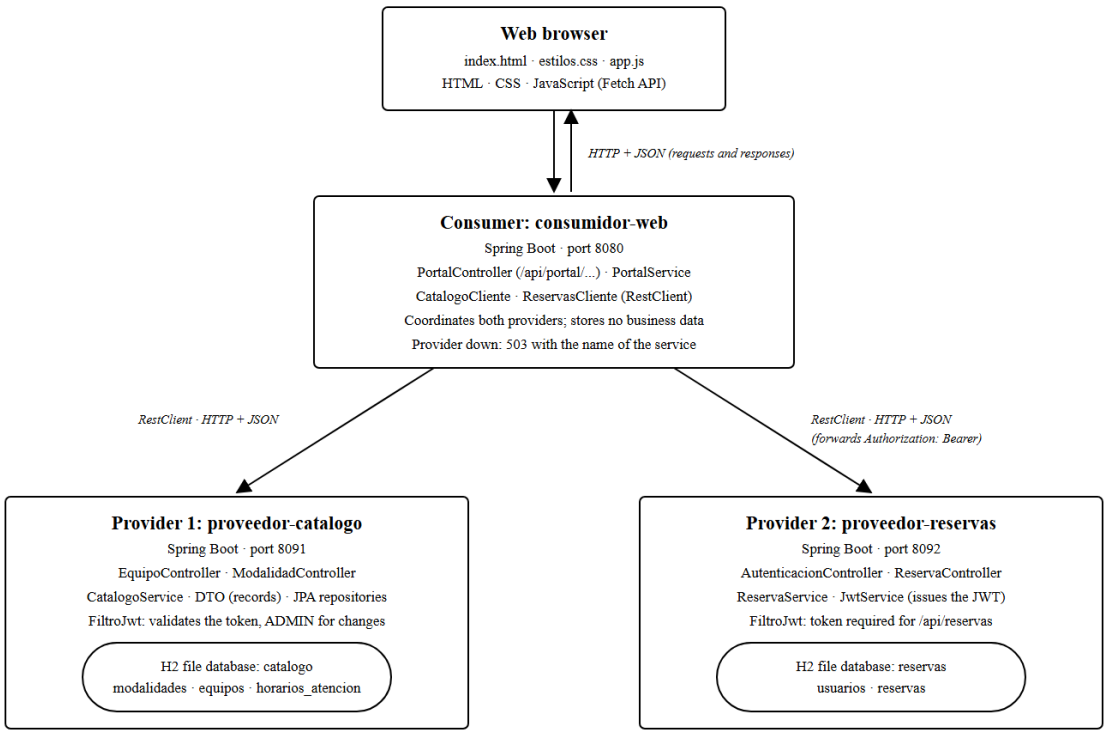
Service-Oriented Architecture

Figure 1 shows the three applications. Each one is a separate Maven project with its own *pom.xml*, process, port, and configuration. The browser only talks to the consumer, and the consumer only talks to the providers through their HTTP interfaces; it never reads or writes the

providers' databases. The provider URLs are properties of the consumer (*app.proveedores.catalogo.url* and *app.proveedores.reservas.url*) that can be changed with the environment variables CATALOGO_URL and RESERVAS_URL.

Figure 1

SOA Diagram of Game Spot



Note. Arrows show the direction of the requests; every response returns along the same path.

Table 3

Responsibilities and Data of Each Application

Application	Responsibility	Data it owns
proveedor-catalogo (8091)	Lists stations, rates, and opening hours; creates, updates, and deletes stations; changes rates. Validates the JWT and requires the ADMIN role for changes.	modalidades, equipos, and horarios_atencion in the H2 database catalogo
proveedor-reservas (8092)	Signs users in and issues the JWT; creates bookings with the block, price, and overlap rules; lists bookings and the occupancy of a day; pays and cancels.	usuarios and reservas in the H2 database reservas

Application	Responsibility	Data it owns
consumidor-web (8080)	Serves the web page; calls both providers with RestClient; coordinates availability and booking; translates provider failures into clear messages.	None; it keeps no business data

API Contract

The contract of a service is what it promises to its clients: the route and method of each operation, the JSON it receives and returns, whether it needs a token, and the status codes it answers. Tables 4 to 6 list the endpoints. All of them exchange JSON, and every error has the same body: a timestamp, the status code, a message in Spanish, and a list of details, for example one per invalid field.

Table 4

Provider 1 Endpoints (Catalog, Port 8091)

Method	Route	Token	Responses
GET	/api/equipos	No	200 list of stations
GET	/api/equipos/{id}	No	200 station; 404 not found
POST	/api/equipos	ADMIN	201 created; 400 invalid data; 404 unknown type; 409 duplicate name
PUT	/api/equipos/{id}	ADMIN	200 updated; 400; 404; 409
DELETE	/api/equipos/{id}	ADMIN	204 deleted; 404
GET	/api/modalidades	No	200 station types with their hourly and half-hour rates
PUT	/api/modalidades/{id}	ADMIN	200 updated; 400 rates outside 1 to 1,000,000; 404
GET	/api/horarios	No	200 opening hours of the seven days

Note. Routes that require a token answer 401 without a token or with an invalid one, and 403 with a customer token.

Table 5

Provider 2 Endpoints (Bookings, Port 8092)

Method	Route	Token	Responses
POST	/api/auth/login	No	200 token; 400 invalid data; 401 wrong credentials
POST	/api/reservas	Yes	201 booking pending payment; 400 invalid block; 409 block taken
GET	/api/reservas/mias	Yes	200 the user's bookings, newest first
GET	/api/reservas/{id}	Yes	200 booking; 403 another user's booking; 404
GET	/api/reservas/ocupacion?equipoId=&fecha=	No	200 occupied intervals of the day, without customer data
PATCH	/api/reservas/{id}/pago	Yes	200 confirmed; 409 not pending or expired
PATCH	/api/reservas/{id}/cancelacion	Yes	200 cancelled; 403; 409 already cancelled or expired

Table 6

Consumer Endpoints (Port 8080)

Method	Route	Providers used	Responses
GET	/api/portal/estado	Both	200 availability of each provider
GET	/api/portal/equipos	Catalog	200; 503 if the catalog is down
GET	/api/portal/horarios	Catalog	200 opening hours sorted by day; 503
GET	/api/portal/equipos/{id}/disponibilidad	Both	200 blocks of the day with their state
POST	/api/portal/login	Bookings	200 token; 401
POST	/api/portal/reservas	Both	201; 400 outside opening hours; 401; 409; 503
GET	/api/portal/reservas/mias	Bookings	200
PATCH	/api/portal/reservas/{id}/pago and /cancelacion	Bookings	200; 409
PATCH	/api/portal/equipos/{id}/estado	Catalog	200; 401; 403

Example Exchange

The consumer creates a booking in Provider 2 with the following request, after reading the station name and both rates from Provider 1. The booking lasts 90 minutes, so its price is one hour at 5,000 plus a half hour at 3,000:

```
POST http://localhost:8092/api/reservas
Authorization: Bearer eyJhbGciOiJIUzM4NCJ9...
{ "equipoId": 2, "nombreEquipo": "Xbox Series S #1",
  "tarifaHoraPesos": 5000, "tarifaMediaHoraPesos": 3000,
  "inicio": "2026-11-19T00:00:00Z", "duracionMinutos": 90 }

201 Created
{ "id": 70, "equipoId": 2, "nombreEquipo": "Xbox Series S #1",
  "inicio": "2026-11-19T00:00:00Z", "fin": "2026-11-19T01:30:00Z",
  "precioTotalPesos": 8000, "adelantoPesos": 4000,
  "estado": "PENDIENTE_PAGO", "emailCliente": "laura@gamespot.demo",
  "pagarAntesDe": "2026-11-12T01:19:00Z" }
```

Data Transfer Objects

Every request and response body is a Java record, and input records carry Bean Validation annotations. DTOs keep the entities hidden: a client never sees the database structure, only the fields of the contract. Table 7 lists the main ones.

Table 7

Main Data Transfer Objects

DTO	Application	Fields and types	Use
EquipoRequest	Catalog	nombre: String (3 to 60); modalidadId: Long; estaDisponible: Boolean	Create or update a station
EquipoResponse	Catalog	id: Long; nombre: String; modalidadId: Long; modalidad: String; tarifaHoraPesos, tarifaMediaHoraPesos: int; estaDisponible, sePuedeReservar: boolean	Station returned by the API
TarifaRequest	Catalog	tarifaHoraPesos, tarifaMediaHoraPesos: Integer (1 to 1,000,000); estaActiva: Boolean	Change the rates of a type
HorarioResponse	Catalog	diaSemana: short; horaApertura, horaCierre: LocalTime	Opening hours of a day
LoginRequest	Bookings	email: String (e-mail); password: String	Sign in
TokenResponse	Bookings	token, tipo, nombre, email, rol: String; expiraEn: Instant	Issued token

DTO	Application	Fields and types	Use
CrearReservaRequest	Bookings	equipoId: Long; nombreEquipo: String; tarifaHoraPesos, tarifaMediaHoraPesos: Integer; inicio: Instant; duracionMinutos: Integer (30 to 240)	Create a booking
ReservaResponse	Bookings	id, equipoId: Long; nombreEquipo: String; inicio, fin: Instant; precioTotalPesos, adelantoPesos: int; estado (including EXPIRADA); emailCliente: String; pagarAntesDe: Instant	Booking returned by the API
ReservaPortalRequest	Consumer	equipoId: Long; fecha: LocalDate; hora: LocalTime; duracionMinutos: Integer	Booking form of the web page
DisponibilidadResponse	Consumer	equipo; fecha: LocalDate; atiende: boolean; bloques: list of {inicio, fin, estaLibre, motivo}	Availability blocks
ApiError	All	timestamp: Instant; status: int; mensaje: String; detalles: List<String>	Every error response

Application Design

Controller, Service, and Repository Layers

The three applications follow the same layered structure. Controllers, such as *EquipoController*, *ReservaController*, and *PortalController*, map routes and HTTP methods, validate the input with *@Valid*, and delegate to a service; they contain no business rules. Services, such as *CatalogoService*, *ReservaService*, and *PortalService*, hold the rules and run inside transactions. In the providers, services read and write data only through Spring Data JPA repositories, which play the role of the Data Access Object pattern: *EquipoRepository*, for example, declares *existsByNombreIgnoreCase*, and *ReservaRepository* declares the JPQL query *buscarActivasQueSeCruzan*, which finds the active bookings of a station that overlap an interval (*r.inicio < :hasta and r.fin > :desde*): confirmed ones and pending ones still within their 15-minute payment window. A global exception handler in each application turns exceptions into *ApiError* responses with the right status code.

RestClient in the Consumer

The consumer defines one RestClient per provider in *ClientesConfig*, with the provider's base URL, a connection timeout of 2 seconds, and a read timeout of 5 seconds. *CatalogoCliente* and *ReservasCliente* wrap every call: they build the request, forward the *Authorization* header of the browser when the operation is protected, and convert the JSON response into a DTO. A shared helper, *LlamadaProveedor*, translates failures. A connection error or a timeout becomes a 503 response that names the unavailable service, and an error response from a provider reaches the browser with the same status code and message.

Integrated Operations

The consumer coordinates both providers in two operations. To show the availability of a station on a day, it reads the station and the opening hours from Provider 1 and the occupied intervals from Provider 2, and it marks each 30-minute block as free, occupied, past, or out of service. To create a booking, it follows these steps: (a) the browser sends the station, date, start time, and duration to POST /api/portal/reservas with the token; (b) the consumer reads the station from Provider 1 and rejects it with 409 if it is out of service; (c) it reads the opening hours from Provider 1 and rejects the booking with 400 if it does not fit between 3:00 p.m. and 9:00 p.m.; (d) it sends the station name, both current rates, and the start time converted from Bogotá time to Provider 2 with the same token; (e) Provider 2 validates the block, checks overlaps, computes the price and the 50% deposit, and stores the booking; and (f) the consumer returns the booking to the browser.

Web Interface

The consumer serves *index.html*, *estilos.css*, and *app.js* as static resources. The script uses the Fetch API for every request to */api/portal*, keeps the token in the browser's session storage

after sign-in, and adds it to protected requests. The page shows the opening hours, the stations with both rates (Figures 2 and 3), a guided booking in three steps (station, day and time, and confirmation; Figures 4 and 5), the customer's bookings with buttons to pay the deposit and to cancel (Figure 6), and an administration table for the administrator. It also includes content of the lounge that does not come from the providers: the catalog of games of each console (*datos/juegos.json*), videos of the lounge, a practice version of the weekend "10-second challenge," and the tournaments already held. Error messages from the backend are shown as they arrive, in Spanish, and a status bar shows whether each provider is available.

Data Model and Persistence

DDL

Each provider defines its tables in *schema.sql*, which Spring Boot runs at start-up. The statements use *create table if not exists*, so an existing database keeps its data. Provider 1 has *modalidades* (identity key, unique name, hourly rate and half-hour rate between 1 and 1,000,000, and an active flag), *equipos* (identity key, unique name, foreign key *modalidad_id* to *modalidades*, and an availability flag), and *horarios_atencion* (day of the week from 1 to 7 as the key, and opening and closing times with a check that closing is later). Provider 2 has *usuarios* (identity key, unique e-mail, BCrypt password hash, and a role restricted to CLIENTE or ADMIN) and *reservas* (identity key, foreign key *usuario_id*, station id, station name, start, end, price, deposit, state restricted to PENDIENTE_PAGO, CONFIRMADA, or CANCELADA (EXPIRADA is computed, never stored), creation and cancellation times, and checks that the end is after the start and the deposit does not exceed the price). The station id in *reservas* has no foreign key because stations live in another application with its own database.

DML and CRUD

Each provider loads its initial data with *data.sql*. Provider 1 inserts the five real stations, the three types with their hourly and half-hour rates, and the opening hours of the seven days (3:00 p.m. to 9:00 p.m.) with *merge into*, and Provider 2 inserts three test users. Every insert runs only if the row does not exist, so restarting the application does not duplicate data or undo changes made through the API; a one-time correction updates databases created before the half-hour rate existed. Table 8 shows how each CRUD operation of the API corresponds to SQL.

Table 8

Correspondence Between CRUD, HTTP, and SQL

CRUD	HTTP endpoint	SQL executed through JPA
Create	POST /api/equipos; POST /api/reservas	INSERT INTO equipos ...; INSERT INTO reservas ...
Read	GET /api/equipos; GET /api/reservas/mias	SELECT ... FROM equipos JOIN modalidades ...; SELECT ... FROM reservas WHERE usuario ...
Update	PUT /api/equipos/{id}; PUT /api/modalidades/{id}; PATCH /api/reservas/{id}/pago	UPDATE equipos SET ...; UPDATE modalidades SET ...; UPDATE reservas SET estado = ...
Delete	DELETE /api/equipos/{id}	DELETE FROM equipos WHERE id = ?
Logical delete	PATCH /api/reservas/{id}/cancelacion	UPDATE reservas SET estado = ’CANCELADA’, cancelada_en = ...

Note. Bookings are cancelled instead of deleted because the lounge needs to keep the history of what was reserved and paid, including deposits that are not refunded; a cancelled booking frees its block.

ORM

The providers use JPA with Hibernate to map entities to tables. The *Equipo* entity, for example, is annotated with *@Entity* and *@Table(name = "equipos")*; its *id* field uses *@GeneratedValue(strategy = IDENTITY)*; and its *modalidad* field uses *@ManyToOne* with

@JoinColumn(name = "modalidad_id"), so loading a station also loads its type and rates. Enumerations such as *EstadoReserva* are stored as text with *@Enumerated(EnumType.STRING)*. Hibernate runs with *ddl-auto: validate*: it does not create tables, it only checks at start-up that the entities match the tables created by *schema.sql*, which is the single source of truth of the schema.

Persistence Mechanism

Each provider uses an H2 database stored in a file (*./data/catalogo* and *./data/reservas*, relative to the working directory of the application), as allowed by the course guide, so the data survives a restart and no database server is needed. The team verified this by creating and paying a booking, stopping both providers, starting them again, and finding the booking still confirmed. The automated tests use in-memory H2 databases created from the same scripts. The monolithic version of the project uses PostgreSQL on Supabase; the service-oriented version does not use it.

Authentication and Authorization With JWT

Provider 2 authenticates users. POST /api/auth/login compares the password with its BCrypt hash and returns a JSON Web Token (Jones et al., 2015) signed with HMAC-SHA. The token carries the e-mail as subject, the role and name as claims, and an expiration of two hours. The browser keeps the token and sends it as *Authorization: Bearer* in every protected request; the consumer forwards it unchanged to the provider that owns the operation.

Both providers validate tokens with a servlet filter and the same secret, read from the environment variable JWT_SECRETO and never written in the repository. In Provider 2, every route under */api/reservas* requires a valid token except the public occupancy query, and the service also checks that a customer only reads, pays, or cancels their own bookings (403 otherwise). In Provider 1, queries are public, and creating, updating, or deleting requires a token whose role is ADMIN. A request without a token or with an invalid or expired one receives 401,

and a valid token without the required role receives 403. The consumer does not validate tokens itself; it relies on the providers, which are the owners of the protected resources.

Test Plan and Results

The test plan has three levels. First, each project has automated integration tests that check every acceptance criterion and are run with *./mvnw test*. Second, a Postman collection exercises the real applications over HTTP. Third, a manual end-to-end test uses the web page, including a stopped provider. Every automated case has a code (CP-36 for the catalog, CP-37 for bookings, and CP-38 for the consumer) that also appears in the test name.

Automated Tests

The provider tests start the whole application with *@SpringBootTest* and an in-memory H2 database, call the endpoints through MockMvc with real tokens, and roll back each test with *@Transactional*. The consumer tests replace the providers with Mockito doubles to check the coordination logic, and a separate class uses a mock HTTP server and a closed port to check the RestClient calls, the forwarded token, the translation of provider errors, and the behavior with a stopped provider. Tables 9 and 10 summarize them. On November 11, 2026, the 29 tests passed with no failures.

Table 9

Automated Tests by Project

Project	Test classes	Tests	Result
proveedor-catalogo	CatalogoTests (CP-36a to CP-36h)	8	0 failures
proveedor-reservas	ReservasTests (CP-37a to CP-37k)	11	0 failures
consumidor-web	PortalTests and ClientesHttpTests (CP-38a to CP-38j)	10	0 failures
Total		29	0 failures

Table 10

Automated Test Cases

Case	What it checks
CP-36a	Lists the five real stations with their type, both rates, and state
CP-36b	Gets a station by id and answers 404 if it does not exist
CP-36c	Full CRUD of stations with an administrator token
CP-36d	Validates the data (400) and rejects duplicate names (409)
CP-36e	Without a token or with an invalid one answers 401; a customer gets 403
CP-36f	Changes the rates of a type and the stations reflect them
CP-36g	Returns the opening hours of the seven days, all from 3:00 to 9:00 p.m.
CP-36h	data.sql corrects an old database once and then respects the changes made through the API
CP-37a	Login returns a JWT with the role; wrong credentials answer 401
CP-37b	Creates the booking in PENDIENTE_PAGO with the price and a 50% deposit
CP-37c	Misaligned blocks, invalid durations, or past blocks answer 400
CP-37d	A block that overlaps another active booking of the same station answers 409
CP-37e	Each customer sees only their bookings; the occupancy is public and has no customer data
CP-37f	Paying confirms the booking; paying it again answers 409
CP-37g	Cancelling is logical and frees the block; another customer gets 403
CP-37h	Protected operations without a token or with an invalid one answer 401
CP-37i	Price: each hour at the hourly rate and the remaining half hour at its own rate
CP-37j	An unpaid booking expires after 15 minutes: it is listed as EXPIRADA, frees the block, and paying or cancelling it answers 409
CP-37k	Within 15 minutes the booking stays pending, shows its payment deadline, and blocks the slot; once paid it never expires
CP-38a	Lists the stations of the catalog provider
CP-38b	Availability combines the opening hours with the occupancy of Provider 2
CP-38c	Booking takes the name and both rates from the catalog, creates it in Provider 2, and returns the payment deadline
CP-38d	Outside the opening hours 400, station out of service 409, invalid data 400, no session 401
CP-38e	With the catalog stopped, the consumer answers 503 and keeps serving the rest
CP-38f	The web page is served by the consumer
CP-38g	RestClient requests GET /api/equipos from the catalog and reads the JSON
CP-38h	The token is forwarded to Provider 2 and its 409 arrives with its message
CP-38i	With a provider stopped (closed port), the call becomes a 503
CP-38j	The opening hours come sorted by day and answer 503 with the catalog stopped

Postman

The collection *soa/postman/GameSpot-SOA.postman_collection.json* has 40 requests in five folders: authentication, Provider 1, Provider 2, consumer, and stopped provider. It stores the tokens and the created identifiers in collection variables and computes tomorrow's date in Bogotá time before the booking requests, so it can be run repeatedly: it deletes the station it creates, cancels its bookings, and restores the station it takes out of service. Every request has a test that checks its status code, and some also check values, such as the 2,500-peso deposit of a one-hour booking and the payment deadline of a new booking. The first four folders run with the three applications started, and the fifth one after stopping Provider 1.

End-to-End Test

On November 11, 2026, the team started the three applications and used the web page as a customer. Figure 2 shows the home page with the opening hours and the figures read from the providers, and Figure 3 the signed-in customer and the stations read from Provider 1 with both rates. In the guided booking, the customer chose the Xbox Series S #1, the next day, and 90 minutes; Figure 4 shows the free blocks from 3:00 p.m. to 8:30 p.m. and the price of 8,000 pesos with a 4,000-peso deposit. Figure 5 shows the booking created through the integrated operation, with the 15-minute payment deadline. Figure 6 shows it confirmed after paying the deposit, next to a second booking that is still pending and shows the time by which it must be paid; cancelling the confirmed one asks for confirmation and warns that the 4,000-peso deposit is not refunded.

Figure 2

Home Page With the Opening Hours and the Figures of the Lounge

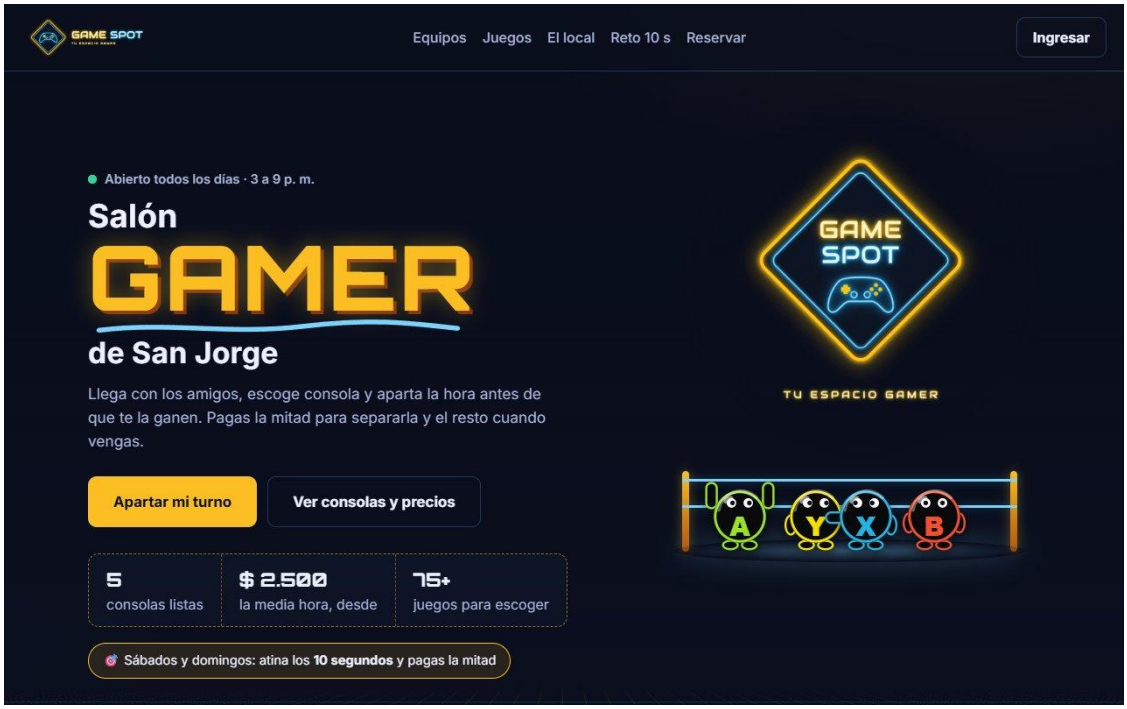


Figure 3

Signed-In Customer and Stations From Provider 1



Figure 4

Availability of a Day: Opening Hours (Provider 1) and Occupancy (Provider 2)

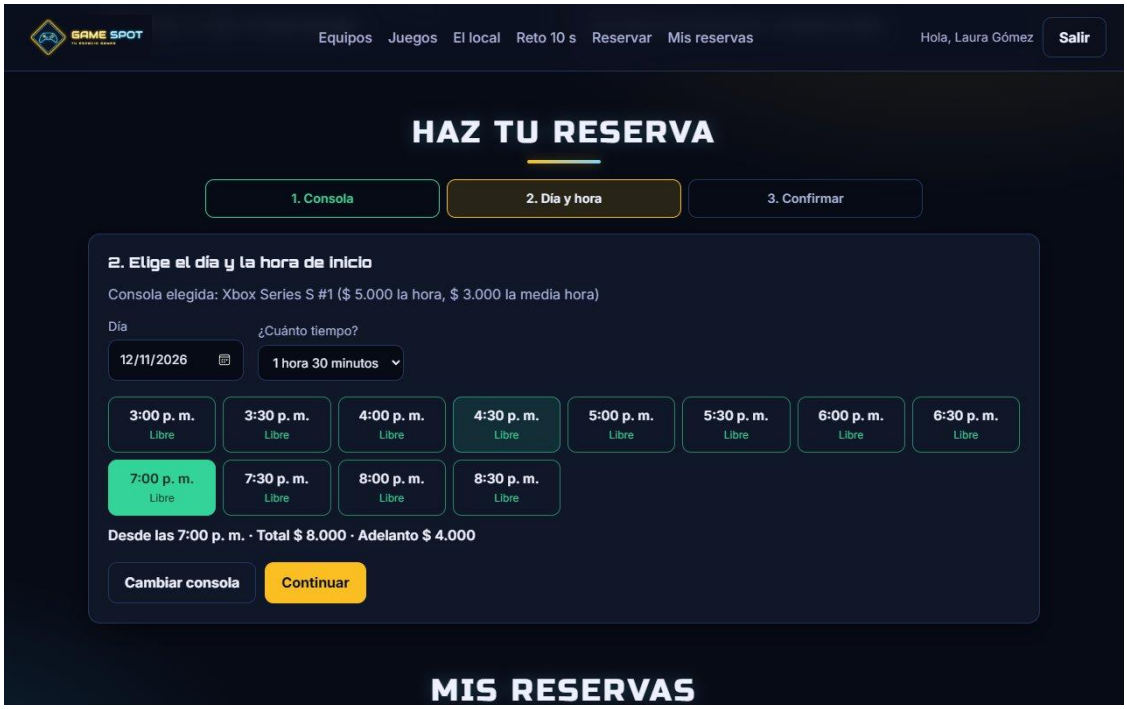


Figure 5

Booking Created With Both Providers

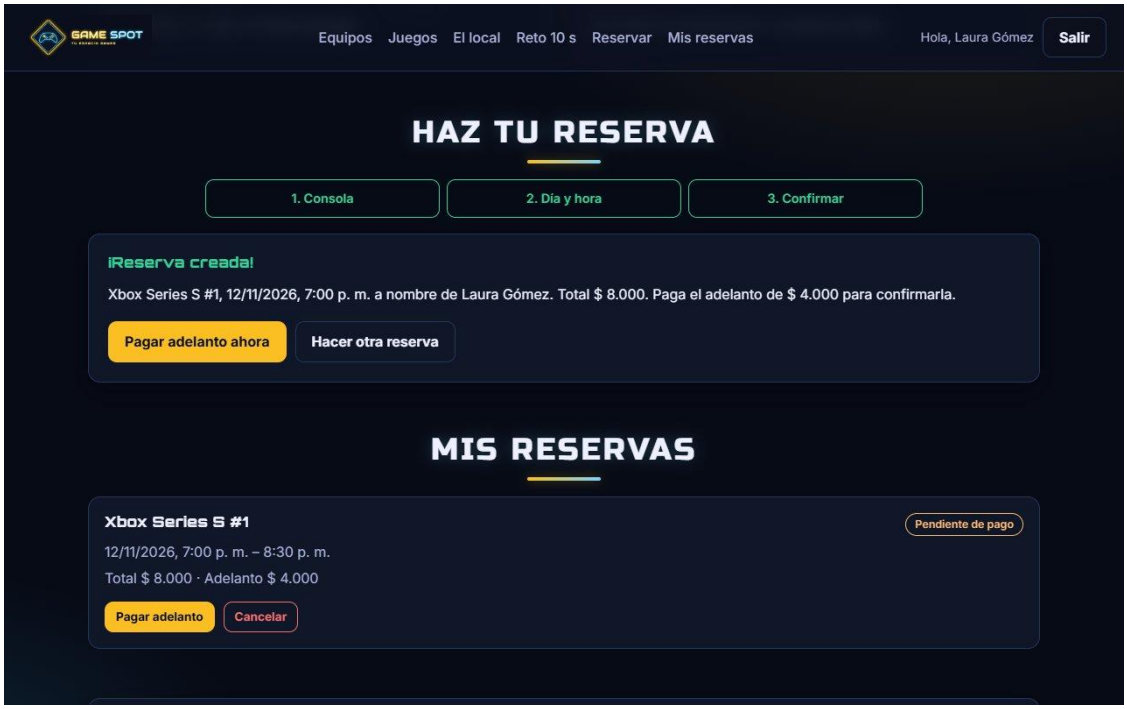
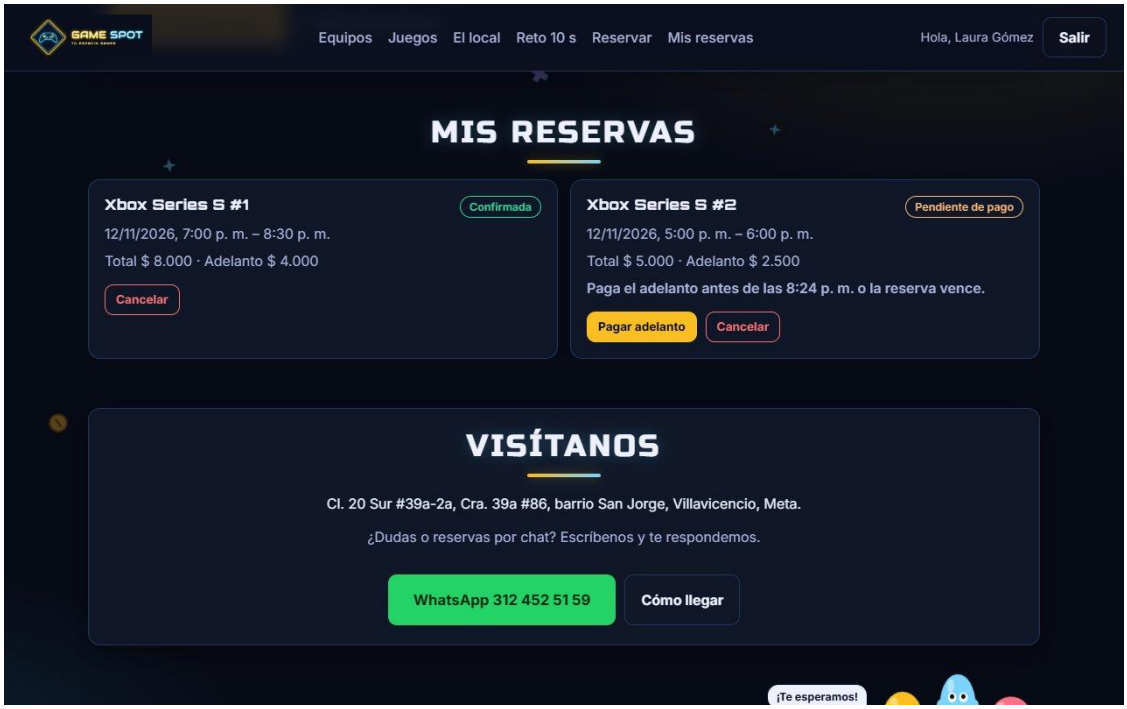


Figure 6

Customer's Bookings: One Confirmed and One Pending With Its Payment Deadline



Stopped Provider

With the three applications running, the team stopped Provider 1. As Figures 7 and 8 show, the page kept working: the stations section shows the message returned by the consumer, which answered GET /api/portal/equipos with status 503 and the text that the catalog service is not available, instead of an unhandled error; and the customer's bookings, which come from Provider 2, are still listed and can be cancelled.

Figure 7

Stations Section With the Catalog Provider Stopped

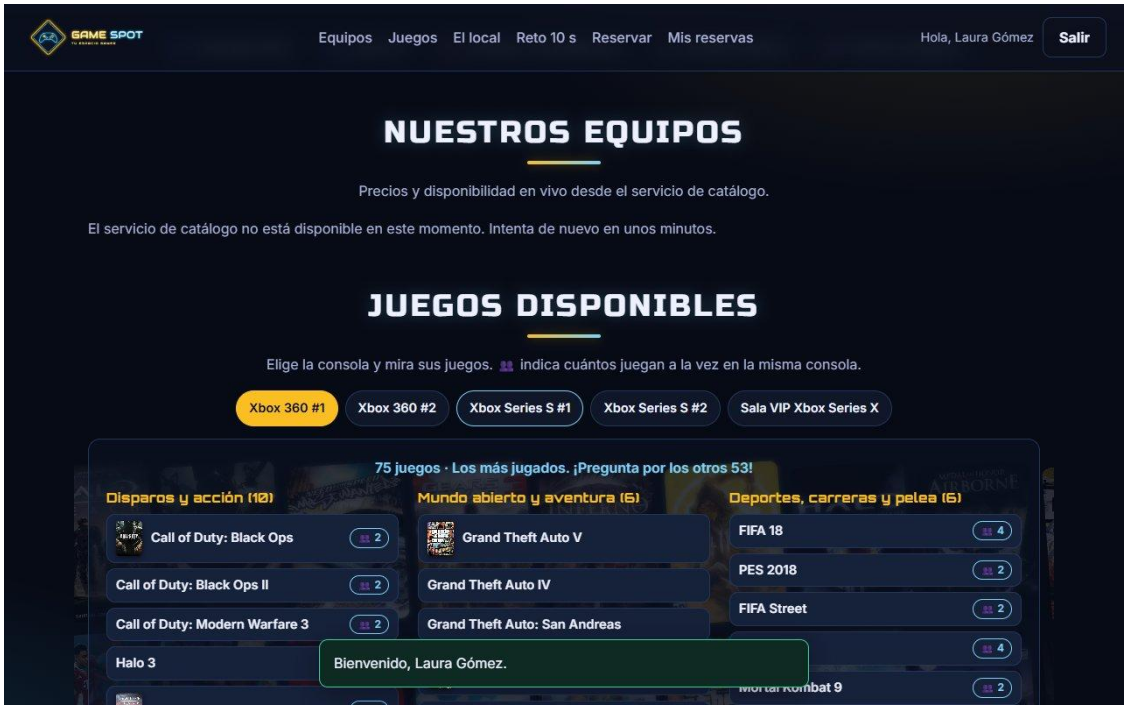
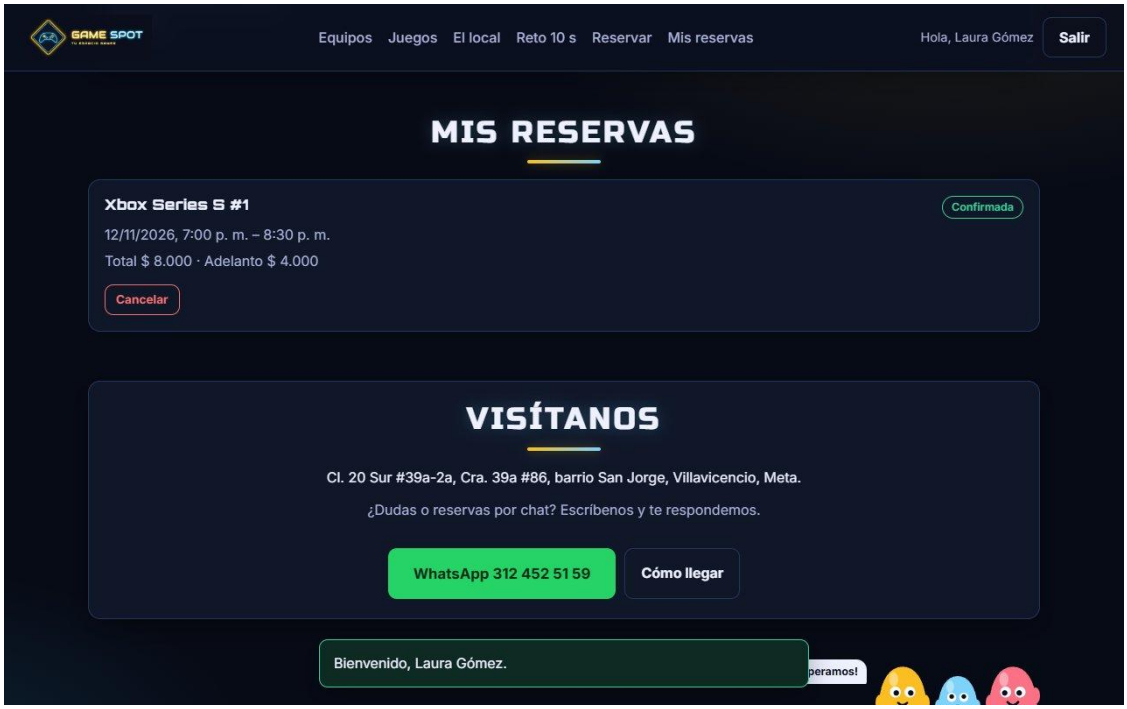


Figure 8

Customer's Bookings Still Available With the Catalog Stopped



Execution Guide

The applications need JDK 21. Docker and an external database are not needed. Before starting the providers, define the environment variable JWT_SECRETO with the same value of at least 32 characters in both of them, for example in the run configurations of IntelliJ IDEA. Then start, in this order, *soa/proveedor-catalogo*, *soa/proveedor-reservas*, and *soa/consumidor-web* with *./mvnw spring-boot:run* (or *.\mvnw.cmd* in PowerShell), and open http://localhost:8080. The H2 files are created in the working directory of each application, so it should not be changed between runs. Table 11 lists the configuration of each application.

Table 11

Configuration of Each Application

Application	Port	Main settings
proveedor-catalogo	8091	JWT_SECRETO; H2 file at ./data/catalogo
proveedor-reservas	8092	JWT_SECRETO; H2 file at ./data/reservas; tokens valid for 2 hours
consumidor-web	8080	CATALOGO_URL (default http://localhost:8091); RESERVAS_URL (default http://localhost:8092); timeouts of 2 and 5 seconds

Note. Test accounts: admin@gamespot.demo with password AdminDemo123, and laura@gamespot.demo or andres@gamespot.demo with password ClienteDemo123. They are demonstration data, not real credentials.

Conclusions, Difficulties, and Lessons Learned

The solution meets the minimum architecture of the course: two providers with different capabilities, a consumer that integrates them with RestClient, a web interface with Fetch, persistence with DDL, DML, repositories, and ORM, and JWT protecting the modifying operations. The integrated booking shows the main benefit of the design: the consumer composes the catalog and the bookings without either provider knowing about the other, and a failure in one provider affects only the features that depend on it.

The main difficulty was that the team started from a monolithic system with a single database that enforced the booking rules with constraints and triggers. Splitting it meant deciding which application owns each piece of data and moving some rules to the services, such as the overlap check, which the monolith enforced with a PostgreSQL exclusion constraint that H2 does not offer. A second difficulty was security across applications: both providers had to validate the same tokens without calling each other, which the team solved with a shared secret provided through an environment variable. A third one was making failures visible to the user, which required timeouts in the RestClient and a uniform error body in all three applications.

Finally, the progress review showed that some business rules were still open; the team fixed a single schedule, implemented the non-refundable deposit and the 15-minute expiration, and left extra time for after the main flow.

The team learned that a service contract has to be fixed before the consumer is built, that business rules must be closed with the client before they are coded, that a consumer must treat every remote call as something that can fail, and that copying the data a service needs, such as the station name and rates in a booking, keeps the services independent at the cost of some duplication.

References

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Doctoral dissertation, University of California, Irvine]. https://ics.uci.edu/~fielding/pubs/dissertation/top.htm

H2 Database Engine. (n.d.). *H2 database engine*. Retrieved November 10, 2026, from https://www.h2database.com/html/main.html

Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). Internet Engineering Task Force. https://doi.org/10.17487/RFC7519

Postman. (n.d.). *Postman learning center*. Retrieved November 10, 2026, from https://learning.postman.com/docs/

Spring. (n.d.). *REST clients*. Spring Framework reference documentation. Retrieved November 10, 2026, from https://docs.spring.io/spring-framework/reference/integration/rest-clients.html

Appendix

Statement on the Use of Artificial Intelligence

The team used an AI coding assistant (Claude, by Anthropic) to support the writing of code, tests, the Postman collection, and this document. The team defined the business idea, the division into services, the business rules, and the design decisions, reviewed every pull request, and verified the results reported here by running the tests, the Postman collection, and the applications.